CE / CS / SE 3354 · SOFTWARE ENGINEERING · FALL 2026

Project Deliverable P1

Proposal and Requirements — Team Template

Weight 4% · Due Sunday 18 October 2026, 11:59 PM Central · Submit one PDF per team on Canvas

Field	Your entry
Project title	Cooket
Team number and section	Team: 1 Section: 005
Team members (name · net ID)	Hamza Munis · hxm240022 Krish Bharal - kxb230038 David Vu (dmv160030) Raagavika Govindarajan – rxg240015 Siddharth De – SXD230138 Gabriel Aguilar-Vega - gja230002
Repository URL	https://github.com/zxahamza20/CS3354-Project-Cooket
Document version	0.1
Date	10/15/2026

0 Revision History and Contribution Record

Both tables below are graded. The contribution record carries the individual 20% of this milestone and must be consistent with your repository history.

Version	Date	Author	Summary of change
0.1	10/15/2026		Initial baseline for P1

0.1 Individual contribution record

Team member	What this person produced for P1	Evidence in the repository
Hamza Munis	Created Repo, Stakeholders, purpose, assumptions	Initial Creation
Krish Bharal	Scope, User Requirements, Definitions, Stakeholders, Constraints, Feasibility Summary	No Code/Repo Not Used
Siddharth De	Functional Requirements, Quality Requirements, Constrain Requirements	No Code/Repo Not Used
Gabriel Aguilar-Vega	Context Diagram, References	Repo not needed
David Vu	Risks, A4/A5 in Assumptions/Dependencies, Repository/Tooling, Open Questions	Repository had not been used.
Raagavika Govindarajan	Initial Use Case, Use-Case Diagram, Expanded Use Case, Requirement-to-Use-Case Traceability	Repository has not been used.

1 Introduction

1.1 Purpose of this document and of the system

The purpose of this document is to define the baseline with functional and non-functional requirements for the Cooket recipe recommendation system. It lays out the technical boundaries, user roles, and operational constraints and serves as the formal design agreement between the project development team and the course instructors before the architectural and implementation decisions.

The system solves the problem of finding the right recipes, based on the available ingredients. Manual searching for recipes is very inefficient and can cause decision fatigue because it often displays a lot of non-instructional data and half the time the recipe includes ingredients which are not even available at hand, because it assumes the pantry is stocked. Currently, home cooks like students with isolated or leftover ingredients must browse multiple recipe sources, manually cross-reference ingredient lists, and guess how well a dish matches their immediate cravings. Cooket automates web recipe scraping and provides a structured recommendation system where users can query recipes by available ingredients, having a flexible or limited pantry and rank matching options across different culinary criteria such as flavor profiles, textures, and target food categories.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
Taking inputs from users in text form	Automatically get ingredients from image
Get and filter results of web-scraping recipes	Generate new recipes
Suggest recipes based on inputs	Handle payments
Automatically be able to buy new ingredients for recipe	

Why the exclusions were excluded:

Automatically getting ingredients from images is out of scope because it is unneeded for the project goal. We will not generate new recipes, since AI generated recipes cannot be fully trusted. We're not going to handle payments due to security risks and liability. Additionally, we want our product to be usable without an account.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
Web Scraping	The automated process of collecting recipe information, including ingredients, instructions, and other relevant details, from websites.
Strict Ingredient Mode	A search option that only recommends recipes that can be made using the ingredients the user has specified.
Flexible Ingredient Mode	A search option that allows recipes requiring additional ingredients beyond those specified by the user.

Term or abbreviation	Meaning as used in this project
Match Score	A value representing how closely a recipe matches the user's ingredients and preferences, used to rank recommendations.
Recipe Database	A collection of recipes and associated information retrieved from online sources for searching and recommendations.

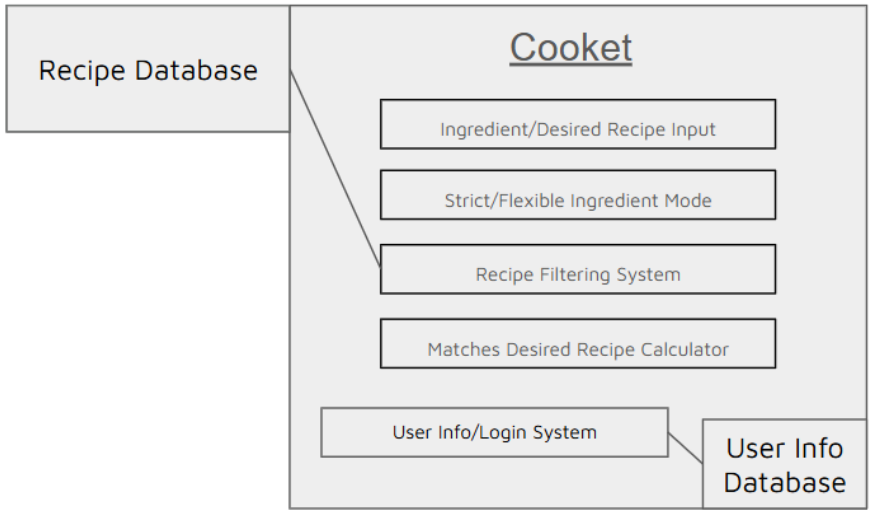
1.4 References

[R1]

No References Used here

2 Overall Description

2.1 Product perspective and context



The recipe database sits outside our system as we will have a web scraper search for recipes throughout the internet to put into the system. It's the only system we will not create with every other system/database being made/maintained by us while also getting recipes from the database if the system requires it.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
User	They use the system, and want the system to be the best, as in report any bugs or glitches and are the reason advertisers sponsor us	A working system that can get them the recipes they want based on their tastes.	Interviewed Baking club members and students interesting in cooking

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Maintainer	Operates, updates, and monitors the scraper, parser, and recipe database	A system can easily be adjusted, has consistent nomenclature, easy to iterate on and deploy.	We assumed based experience as maintainer
Advertisers	The sponsors who pay for the system	The ability to promote their own product and get users to engage with it or get their information	We assumed that's what advertisers would want

2.3 User characteristics

The users are people who want to look up (and usually cook) recipes tailored to themselves. They can be assumed to know what ingredients they have. If they're wanting a specific texture or flavor, they would know that as well. They also can be assumed to know what type of food they want, if any. The only constraint on the users is that the system requires internet and access to the sites we will be web-scraping, so some locations will have worse recipe results.

2.4 Constraints

"We will not handle payments", this is a constraint due to risk of data breaches and overall estimated workload would push the project to take longer than available time.

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	Standard ingredient naming conventions so that ingredients input by users can be matched against the scraped recipe text	Searches will yield false negative zero match results for recipes that list synonyms or modified names
A2	Target recipe websites maintain a reasonable and consistent markup and standard schema structures during our scraping cycles	The extraction pipeline will fail to parse ingredients and instructions, producing stale, corrupt, or incomplete records which would require manual parsing for the affected sites
A3	Target culinary websites remain online, accessible over standard HTTPS, and do not enforce strict IP anti-bot/CAPTCHA firewalls against scraper requests.	Automated recipe ingestion will halt or return HTTP 403/429 status codes, preventing database refreshes and new recipe discovery.
A4	Supabase backend services and database APIs stay online and accessible over our network connection.	The app won’t be able to save or pull scraped recipes, breaking our search and recommendation tools.
A5	Our recipe database storage and query volume stay within Supabase’s free-tier limit.	Database requests might get throttled or blocked, causing recipe searches to time out or fail for users.

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible	3 team members have experience in ML, and one has done work with web scraping see https://github.com/Krish54491
Operational	Feasible	After doing some interviews with cooking and baking inclined students, most students said they would use a product like this, especially since it's free
Schedule	Feasible	Most of our team has experience with the technologies used and the bulk of the work will be done in the backend which is easier to test
Economic / resource	Feasible	There is no Economic / resource being used other than our laptops and time

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-01	The system shall allow users to input their ingredients into the search tab	MUST	MAINTAINER	A tester can confirm this by inputting a basic list of 5 ingredients into the search tab that is understood by the system
FR-02	The system shall be able to returns multiple meal recipes made with the user's ingredients	MUST	MAINTAINER	A tester can confirm this by inputting the base ingredients for simple recipe (like a PB&J sandwich), and check if the system outputs that as a result
FR-03	The system can be able to sort by popular recipes	COULD	USER	A tester can confirm this by inputting their ingredients and getting an organized output of popular recipes
FR-04	The system includes accurate measurements for recipes in any measurement quantities	MUST	MAINTAINER	This can be tested by inputting ingredients and quantities via the users preferred measurement quantities and seeing if the output follows the same measurements
FR-05	The system should allow the user to save their favorite recipes	SHOULD	USER	A tester can confirm this by selecting the favorite button for a certain recipe and accessing that on the favorites tab
FR-06	The system can provide recommend recipes based on the types of foods/ingredients the user frequently has or uses	WON'T	ADVERTISER	A tester can confirm this by favoriting a few recipes, and then checking to see if the system offers any recommended recipes based on their favorites
FR-07	The system can provide a video of the recipe being made (if the recipe is from a video platform like YouTube)	COULD	USER	A tester can confirm this by selecting a recipe and checking to see if a video of it being made is linked in its description
FR-08	The system can allow you to post your own recipes that others can see/get recommended	WON'T	ADVERTISER	A tester can test this by posting their own recipe and check if they receive other user made recipes in the recommendations

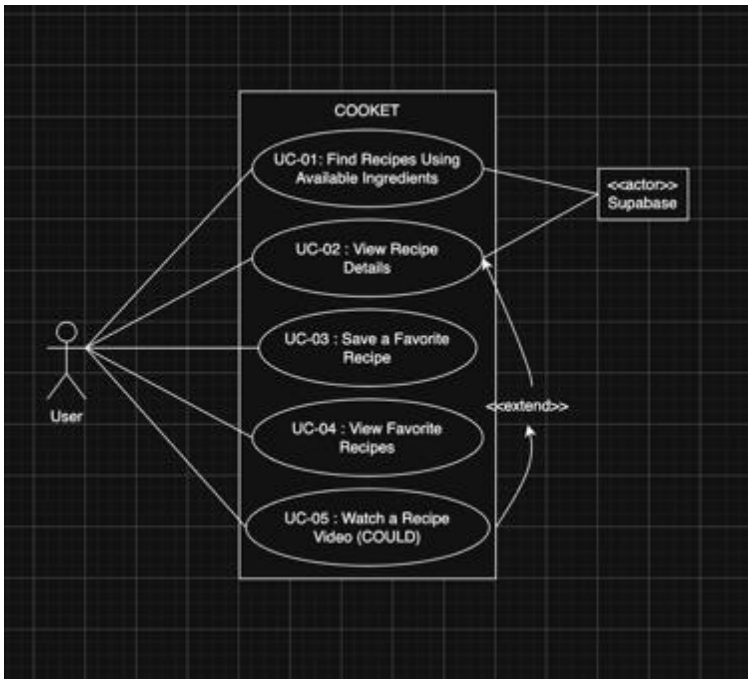
3.2 Quality requirements

ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	The system shall return recipe search results within 3 seconds for at least 95% of searches containing 1–10 ingredients	MUST	USER	This can be verified by performing 100 searches using different ingredient combinations and verifying that at least 95 finishes within 3 seconds
QR-02	The system shall allow at least 90% of first-time users to complete an ingredient search without assistance within 60 seconds	SHOULD	USER	This can be verified by conducting a usability test with 10 first-time users and verify that at least 9 complete the task within 60 seconds
QR-03	The system shall correctly identify at least 90% of ingredient matches between user inputs and recipe ingredient lists in a test set of 100 ingredient comparisons	MUST	MAINTAINER	This can be verified by compare the system's ingredient matching results against 100 manually verified ingredient comparisons and calculate accuracy
QR-04	The system shall use HTTPS for 100% of communication between the user's browser and the application server	MUST	USER	This can be verified by inspecting network requests using browser developer tools and verify that all browser-to-server requests use HTTPS

3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system shall retrieve existing recipes rather than generate new recipes using AI	PROJECT SCOPE	AI-generated recipes are excluded because we want to provide recipes from existing sources rather than risk generating unreliable cooking instructions
CON-02	The system shall allow users to search for recipes without requiring account registration	PROJECT SCOPE	users should be able to access the application's primary functions without creating an account for convenience and keeping users on the app

3.4 Initial use cases



Expanded use case — copy this block for each of your two

Field	Content
Identifier and name	UC-01 - Find Recipes Using Available Ingredients
Primary actor	User
Supporting actors	Supabase database service
Preconditions — what must be true before it starts	The user can access Cooket. The recipe database is available and contains recipe records. No account is required.
Trigger — what starts it	The user submits their available ingredients to search for recipes.
Main success scenario — numbered steps	1. The user enters their available ingredients. 2. The user submits the search. 3. Cooket processes the ingredient input. 4. Cooket retrieves matching recipes from the database. 5. Cooket displays the matching recipes. 6. The user reviews the results and identifiers recipes of interest.
Alternative and exception flows	2a. Empty input: Cooket asks the user to enter ingredients before searching. 4a. No matching recipes: Cooket displays a no-results message and allows the user to revise the search. 4b. Database unavailable: Cooket reports the search could no be completed and allows the user to retry. 5a. Popularity sorting: If this COULD feature is implemented, the user can request that results be sorted by popularity.
Postconditions — what is true when it succeeds	Matching recipes have been displayed for the ingredients submitted by the user.
Requirements realized	FR-01, FR-02; FR-03 for optional popularity sorting.

Field	Content
Identifier and name	UC-03 – Save a Favorite Recipe
Primary actor	User

Field	Content
Supporting actors	To be confirmed once favorites storage is decided.
Preconditions — what must be true before it starts	The user is viewing a recipe in Cooket. The favorites feature is available. No account is required.
Trigger — what starts it	The user selects the recipe's favorite button.
Main success scenario — numbered steps	1. The user selects the favorite button. 2. Cooket checks whether the recipe is already saved. 3. Cooket adds the recipe to the user's favorites. 4. Cooket confirms that the recipe has been saved.
Alternative and exception flows	2a. Already saved: Cooket indicates that the recipe is already in favorites without creating a duplicate. 3a. Saving fails: Cooket displays an error and allows the user to retry.
Postconditions — what is true when it succeeds	The recipe is saved and available through View Favorite Recipes.
Requirements realized	FR-05

3.5 Requirements to use case traceability

Requirement ID	Use case(s) that realize it	If blank — why, and what you will do
FR-01	UC-01 – Find Recipes Using Available Ingredients	-
FR-02	UC-01 – Find Recipes Using Available Ingredients	-
FR-03	UC-01 — Find Recipes Using Available Ingredients, optional popularity-sorting flow	-
FR-04	UC-02 — View Recipe Details	-
FR-05	UC-03 — Save a Favorite Recipe; UC-04 — View Favorite Recipes	-
FR-06	-	Marked WON'T for this release. Retain for future consideration; no current use case is assigned.
FR-07	UC-05 — Watch a Recipe Video	-
FR-08	-	Marked WON'T for this release. Retain for future consideration; no current use case is assigned.

What the matrix revealed, and what you changed

The matrix revealed that all functional requirements, whether planned or proposed for inclusion in this release, have use cases associated with them, and all use cases are associated with a functional requirement. FR-05 was mapped to both the save favorite and view favorites use cases because the two are needed for satisfying its acceptance criterion. FR-06 and FR-08 could not be mapped since they are won't be done for this release.

because of it:

Because of these findings, we linked FR-05 to both saving and viewing favorites and explicitly identified FR-06 and FR-08 as excluded from the current release. Popularity sorting, measurement units, and favorites storage still require clarification.

4 Project Plan

4.1 Team roles and working agreement

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
David Vu	Scope and Feasibility Lead	We discuss the issue as a group and take majority vote if we cannot reach an agreement.
Raagavika Govindarajan	Use case and Traceability	We discuss the disagreements as a team and take majority vote if we cannot reach an agreement.
Krish Bharal	Team Coordinator	We'll discuss the issue as a group, try to compromise, but if there is no compromise, we will take a majority vote.
Hamza Munis	Risk and Repository Lead	We discuss the conflicts as a team, then take a vote if no consensus is made.
Gabriel Aguilar-Vega	Stakeholder & Requirements Lead	Hold a meeting to discuss the disagreement, and if unable to reach a consensus, take a majority vote.
Siddharth De		

4.2 Plan to P2

Date	What will be finished	Owner
10/25	Initial Web scraping recipes without clean up	Krish Bharal
11/3	Frontend that takes in user data and calls backend web scraping	Hamza Munis
11/3	LLM Recommendation	Raagavika Govindarajan
11/1	User login and Registration	Gabriel Aguilar-Vega
11/3	Data storage (database integration)	Siddarth De
11/8	Clean up scripts, and API setup on the cloud	David Vu

4.3 Risks

#	Risk	Likelihood	Impact	Mitigation — action and owner
R1	Supabase Service Outage or Connection Limits: Unexpected cloud downtime or hitting connection limits on Supabase.	Medium	High	Action: Set up local caching or fallback lists so the app still works if the database fails. Owner: Backend Dev
R2	Inconsistent Ingredient Formatting from Web Scraping: Scraped recipes have messy or inconsistent ingredient names that don’t match what users type.	High	High	Action: Write a clean up script to normalize ingredient text before saving it to the database. Owner: Scraper

#	Risk	Likelihood	Impact	Mitigation — action and owner
R3	Target Website Structure Changes: Recipe websites update their HTML or layout, breaking the automated scraping and stopping new data from being collected into the database.	Medium	Medium	Action: Keep parsing logic modular and check the scraper regularly to catch broken layouts early on. Owner: Scraper
R4	Scope Creep on Additional Features: Team members attempting to introduce out of scope features. For example, image recognition or user authentication.	Medium	Medium	Action: Keep everyone aligned on the feature limits so we don't accidentally take on extra work. Anyone wanting to add a new feature has to run it by the team first. Owner: Whole Team
R5	Integration Bottlenecks Between Frontend and Database: Delays in connecting the UI text input with the backend SQL query pipeline.	Medium	High	Action: Define an API/data schema so that the team can develop both the frontend and backend early. Then test using mock data. Owner: Whole Team

4.4 Repository and tooling

Repository URL: https://github.com/zxahamza20/CS3354-Project-Cooket

Database: We are using Supabase as our cloud hosted database to store and manage scraped recipe data. Version control is managed through GitHub.

Branching and Commit: We use feature branches off of main for individual components and keep commit messages short and descriptive.

Issue Tracking: GitHub Issues will be used to track tasks and assign owners to ensure all team members help produce a successful product.

Access: All team members and the professor will have full access to the repository.

4.5 Declaration of AI tool use

Tool	What it was used for	Who used it	How the output was checked
ChatGPT	Definitions	Krish Bharal	I checked the output by seeing what would consider a term not readily understandable then edited it to make sure only necessary terms were listed
Gemini	Checking conventions and standard recipe website formats	Hamza Munis	I verified it against the Schema.org Recipe JSON-LD specification and inspected the raw HTML DOM structures of target sites to confirm that listed properties like recipeIngredient, recipeInstructions, prepTime, cookTime actually exist and parse consistently.

5 Open Questions

#	The question	Who can answer it	Needed by	What you assumed in the meantime
Q1	Database Schema: How should we structure our database tables so we can match what users type against scraped ingredient names without breaking searches?	Scraper or Backend Dev	Before building database queries	We assume standard relational tables linking recipes to ingredient lists.
Q2	Scraper Maintenance: What do we do if a target recipe site changes its layout and our scraper starts pulling garbage data into the database?	Scraper or Backend Dev	Before writing scraper scripts	We assumed the target websites are going to keep consistent HTML layouts.
Q3	Match Scoring: How exactly should we calculate the match score when a recipe needs extra ingredients in flex mode?	Whole Team	Before implementing the search logic	We assumed a simple percentage based on how many user ingredients match the recipe.

6 Baseline Declaration

We confirm that the requirements in section 3 represent our agreed baseline as of the submission date, and that every team member has read this document in full.

Team member	Signature or typed name	Date
David Vu	David Vu	October 16, 2026
Siddharth De	Siddharth De	October 16, 2026
Hamza Munis	Hamza Munis	October 16, 2026
Raagavika Govindarajan	Raagavika Govindarajan	October 16, 2026
Krish Bharal	Krish Bharal	October 17, 2026
Gabriel Aguilar-Vega	Gabriel Vega	October 17, 2026

How P1 Is Marked

P1 is worth 4% of the course grade. Following the syllabus, 80 points are for the shared team artifact and 20 for individual contribution evidence.

Criterion	Points	What full marks look like
Problem, scope and context (1.1, 1.2, 2.1)	10	The problem is a problem somebody actually has. The out-of-scope column is filled and the exclusions are justified. The context diagram's boundary matches what sections 2 and 3 claim.
Stakeholders and elicitation (2.2, 2.3)	10	Stakeholders include at least one who will never use the system. Every need has a stated basis, and assumptions are labelled as assumptions rather than disguised as findings.
Requirements quality (3.1, 3.2, 3.3)	25	Every requirement is one testable “shall” sentence with an identifier, a priority, a source and an acceptance criterion. Quality requirements carry a threshold, a condition and a method. Constraints are distinguished from quality requirements.
Use cases and traceability (3.4, 3.5)	20	Use cases are at actor-goal level, not steps. Two are written out in full, including alternative flows. The matrix is complete and the team says what the gaps mean and what they did about them.
Feasibility and plan (2.6, 4)	10	Each feasibility verdict is backed by evidence a reader can check. The plan has dates and owners. Risks are future events with actionable mitigations.
Open questions and baseline (5, 6)	5	Real open questions, with an owner and a date, and an honest statement of what was assumed in the meantime.
Document quality	—	Guidance boxes removed, sections in order, within the length budget, readable. Marks are deducted here, not awarded.
Individual contribution evidence (0.1)	20	Every member's contribution is distinguishable and traceable to something in the repository a reader can open.